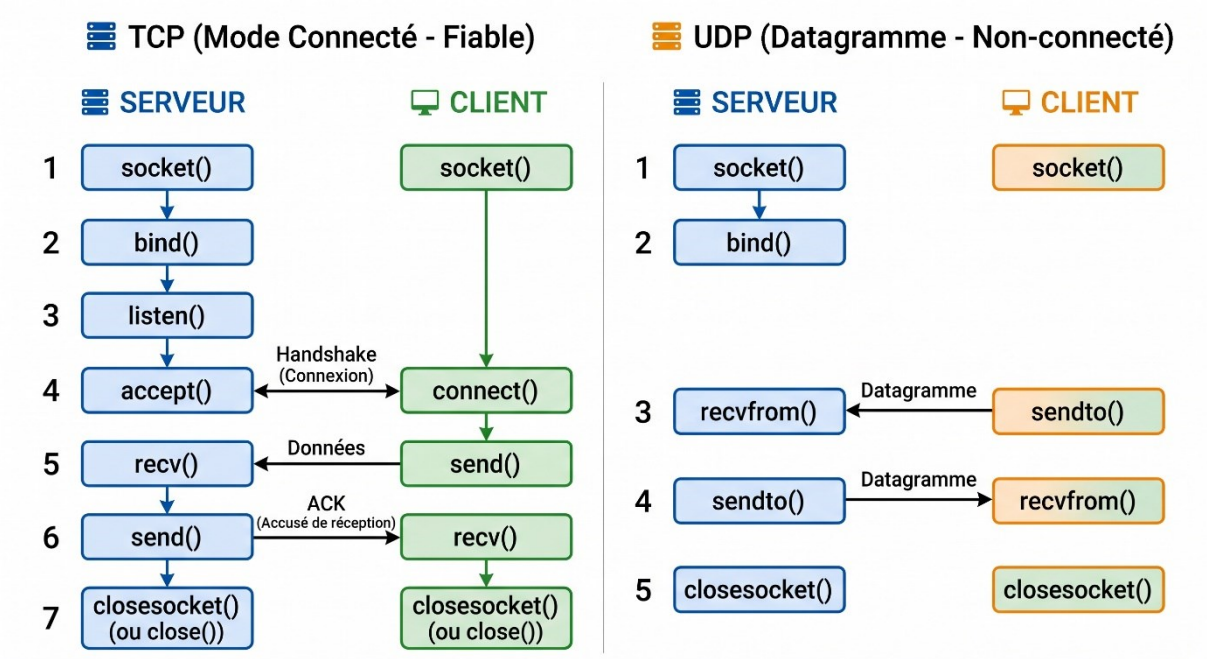
Master Réseaux et Systèmes Distribués USTO-MB
M.N OSMANI

TP03 : Comparaison TCP vs UDP en C (Winsock)

Objectifs Pédagogiques

1. **Distinguer le mode connecté (TCP) du mode non-connecté (UDP) :** Comprendre l'impact de l'absence de négociation préalable (*Handshake*) et d'accusé de réception automatique.
2. **Identifier la différence d'API Berkeley / Winsock** :
   - **TCP** : `SOCK_STREAM`, **cycle:** `listen() / accept() / connect(), transfert via send() / recv().`
   - **UDP** : `SOCK_DGRAM`, pas de `listen()` ni de `connect()`, transfert direct par datagrammes via `sendto() / recvfrom()`.
3. **Mettre en évidence les compromis en systèmes distribués :** Fiabilité et ordonnancement (TCP) vs Vitesse et tolérance aux pertes (UDP, idéal pour streaming/relevés périodiques de capteurs IoT).

Comparatif Architectural TCP vs UDP:



Propriété	TCP (SOCK_STREAM)	UDP (SOCK_DGRAM)
Connexion préalable	Oui (Handshake à 3 voies)	Non (Envoi direct à l'adresse cible)
Fiabilité	Garantie (Réémissions automatiques, ACK)	Non garantie (Paquets perdus non signalés)
Ordonnancement	Garanti (Flux continu ordonné)	Non garanti (Les paquets peuvent arriver désordonnés)
Primitives C	send() / recv()	sendto() / recvfrom()

Code Source de Référence UDP:

Les programmes TCP ayant été abordés au TP01 (server.c et client.c), nous développons ici les équivalents UDP pour permettre aux étudiants de tester et comparer les deux protocoles.

Fichier `server_udp.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>
#define PORT 9090
#define BUFFER_SIZE 1024

int main() {
    WSADATA wsa;
    SOCKET server_sock;
    struct sockaddr_in server_addr, client_addr;
    int client_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE];

    // 1. Initialisation Winsock[cite: 6]
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) {
        printf("[ERREUR] WSAStartup : %d\n", WSAGetLastError());
        return 1;
    }

    // 2. Création de la socket UDP (SOCK_DGRAM)
    server_sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (server_sock == INVALID_SOCKET) {
        printf("[ERREUR] Creation socket UDP : %d\n", WSAGetLastError());
        WSACleanup();
        return 1;
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    // 3. Liaison IP et Port
    if (bind(server_sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) == SOCKET_ERROR) {
        printf("[ERREUR] Bind UDP : %d\n", WSAGetLastError());
        closesocket(server_sock);
        WSACleanup();
        return 1;
    }

    printf("=================================================\n");
    printf("[SERVEUR UDP] En attente de datagrammes sur le port %d...\n", PORT);
    printf("=================================================\n");

    while (1) {
        memset(buffer, 0, BUFFER_SIZE);

        // 4. Réception du datagramme et capture de l'adresse de l'émetteur
        int bytes_received = recvfrom(server_sock, buffer, BUFFER_SIZE - 1, 0,
                                      (struct sockaddr *)&client_addr, &client_len);

        if (bytes_received == SOCKET_ERROR) {
            printf("[ERREUR] recvfrom : %d\n", WSAGetLastError());
            break;
        }

        buffer[bytes_received] = '\0';
        char client_ip[INET_ADDRSTRLEN];
        strcpy(client_ip, inet_ntoa(client_addr.sin_addr));
        int client_port = ntohs(client_addr.sin_port);

        printf("[SERVEUR UDP] Paquet recu de %s:%d -> %s\n", client_ip, client_port, buffer);

        // 5. Réponse optionnelle au client
        const char *ack_msg = "ACK_UDP: Datagramme bien recu";
        sendto(server_sock, ack_msg, (int)strlen(ack_msg), 0,
               (struct sockaddr *)&client_addr, client_len);
    }

    closesocket(server_sock);
    WSACleanup();
    return 0;
}
```

Fichier `client_udp.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>

#define PORT 9090
#define BUFFER_SIZE 1024

int main() {
    WSADATA wsa;
    SOCKET sock;
    struct sockaddr_in dest_addr;
    int addr_len = sizeof(dest_addr);
    char buffer[BUFFER_SIZE];

    // 1. Initialisation Winsock
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) {
        printf("[ERREUR] WSAStartup : %d\n", WSAGetLastError());
        return 1;
    }

    // 2. Création de la socket UDP (SOCK_DGRAM)
    sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock == INVALID_SOCKET) {
        printf("[ERREUR] Creation socket : %d\n", WSAGetLastError());
        WSACleanup();
        return 1;
    }

    // 3. Configuration de l'adresse de destination
    dest_addr.sin_family = AF_INET;
    dest_addr.sin_port = htons(PORT);
    dest_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    printf("==================================================\n");
    printf("[CLIENT UDP] Pret a envoyer des datagrammes au port %d\n", PORT);
    printf("Tapez votre message puis Entree (ou 'exit' pour quitter)\n");
    printf("==================================================\n");

    while (1) {
        printf("> ");
        if (!fgets(buffer, sizeof(buffer), stdin)) break;
        buffer[strcspn(buffer, "\r\n")] = '\0';

        if (strcmp(buffer, "exit") == 0) break;
        if (strlen(buffer) == 0) continue;

        // 4. Envoi direct sans connect() préalable
        int sent = sendto(sock, buffer, (int)strlen(buffer), 0,
                          (struct sockaddr *)&dest_addr, addr_len);

        if (sent == SOCKET_ERROR) {
            printf("[ERREUR] sendto echoue : %d\n", WSAGetLastError());
            continue;
        }

        printf("[CLIENT UDP] %d octets emis avec succes.\n", sent);

        // 5. Lecture de l'accuse eventuel
        memset(buffer, 0, BUFFER_SIZE);
        int received = recvfrom(sock, buffer, BUFFER_SIZE - 1, 0,
                                (struct sockaddr *)&dest_addr, &addr_len);
        if (received > 0) {
            buffer[received] = '\0';
            printf("[CLIENT UDP] Reponse : %s\n", buffer);
        }
    }

    closesocket(sock);
    WSACleanup();
    return 0;
}
```

- Compilez les deux programmes.
- **Terminal 1 :** Démarrez le serveur.
- **Terminal 2 :** Lancez le client et envoyez plusieurs messages.

Expériences Comparatives

Expérience 1 : Lancement du Client sans Serveur actif

- **Procédure TCP (TP01) :** Tenter de lancer `client.exe` sans avoir démarré `server.exe`.
    - *Constat attendu :* `L'appel connect() échoue immédiatement avec l'erreur 10061 (WSAECONNREFUSED / Connexion refusée)`
- **Procédure UDP :** Lancer `client_udp.exe` sans exécuter `server_udp.exe`, puis envoyer un message.
    - *Constat attendu :* `sendto() réussit et indique que les octets ont été envoyés sans lever d'erreur. UDP ne vérifie pas si le récepteur existe ou est en ligne.`

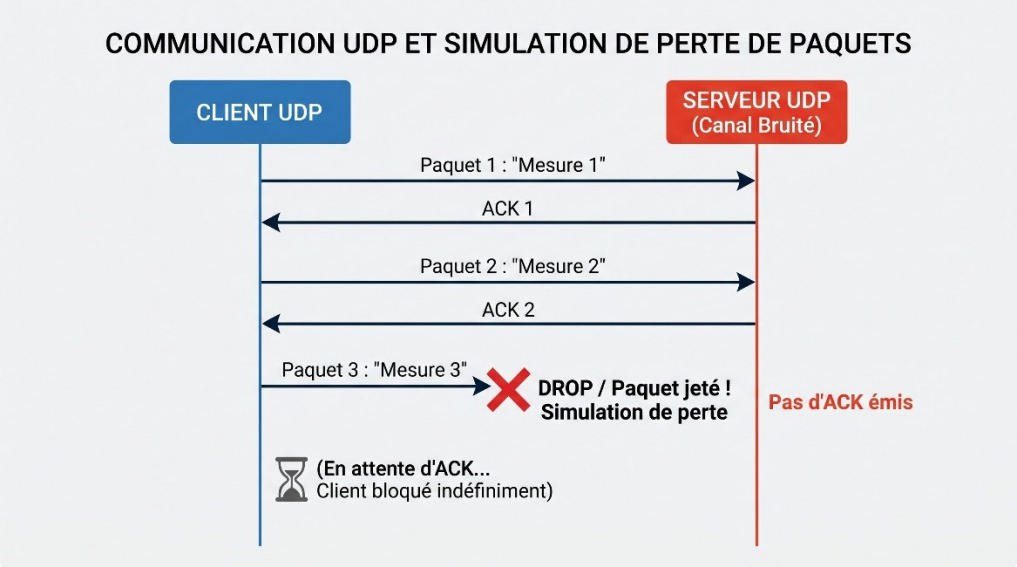
Expérience 2 : Délimitation des Messages (Message Boundaries)

- **TCP :** Si le client envoie 3 messages courts successivement (`send()`, `send()`, `send()`), le serveur peut tout recevoir en un seul bloc agrégé lors d'un unique appel `recv()` (effet *`stream / flux continu`*).
- **UDP :** Si le client envoie 3 datagrammes via `sendto()`, le serveur devra exécuter exactement `3 recvfrom()` pour les récupérer individuellement. Chaque datagramme conserve sa taille et ses frontières.

Exercice d'Application : Simulation de Perte de Paquets

Dans les systèmes distribués réels (comme les réseaux de capteurs IoT agricoles vus au TD1 utilisant LoRa ou Zigbee), le canal de transmission n'est jamais parfait : interférences radio, encombrement des files d'attente ou coupures temporaires entraînent la perte de paquets.

Contrairement à TCP qui retransmet automatiquement les segments manquants sans avertir l'application, **UDP ne fournit aucune garantie de livraison**. Si un paquet est perdu, UDP ne le réémet pas et l'émetteur ne reçoit aucune notification d'erreur.



Objectif du Travail Demandé

On doit introduire un composant simulant un réseau bruité (*packet drop simulator*) dans le serveur UDP :

1. Compter chaque datagramme entrant.
2. Pour **1 paquet sur 3** (c'est-à-dire les paquets n° 3, 6, 9...), simuler une perte réseau :
    - Afficher un avertissement dans la console du serveur.
    - **Ne pas envoyer d'accusé de réception (ACK)** au client.
    - Ignorer le traitement et passer directement à l'attente du paquet suivant.
3. Pour les autres paquets (n° 1, 2, 4, 5...), traiter le message normalement et renvoyer l'accusé de réception.

Modifier `server_udp.c` pour simuler un canal réseau instable. Le serveur doit ignorer arbitrairement 1 paquet sur 3.

```
// Squelette à insérer dans la boucle while(1) de server_udp.c :
static int packet_counter = 0;
packet_counter++;

if (packet_counter % 3 == 0) {
    // [A COMPLETER]
    // Afficher "[RESEAU BRUITE] Datagramme volontairement ignore (DROP) !"
    // Ne pas appeler sendto() et passer directement au tour de boucle suivant
    // ???
}
```

Question 1 : Lorsque vous envoyez le 3ᵉ message depuis le client UDP (`client_udp.exe`), que constatez-vous sur la console du client ? Le client affiche-t-il une erreur réseau (comme un code d'erreur `SOCKET_ERROR`) ?

Question 2 : Si un paquet physique était perdu de la même manière dans une communication TCP (TP01), le programme client se bloquerait-il sur `recv()` de la même façon ? Pourquoi ?

Question 3 : Proposez une solution technique au niveau applicatif (dans votre code C) pour éviter que le client UDP ne reste bloqué à l'infini lorsqu'un paquet est perdu.

- Deux mécanismes doivent être ajoutés :
    - **Un délai d'expiration (Timeout de réception) :** Configurer la socket du client avec l'option `SO_RCVTIMEO` via `setsockopt()`. Si aucun ACK n'est reçu après 2 secondes, `recvfrom()` se débloque en retournant une erreur `WSAETIMEDOUT`.
    - **Un mécanisme de retransmission avec compteur (Stop-and-Wait ARQ) :** Si un timeout survient, le client réémet le datagramme (jusqu'à un maximum de 3 tentatives avant de déclarer le nœud distant injoignable).